📘 AI/ML Internship – Day 83 Notes & Tasks

🟢 Module 10: Project Architecture & System Design

📌 Part 1: Recap of Day 82

Yesterday we learned:

- ✅ What is SRS?
- ✅ Problem Statement
- ✅ Objectives
- ✅ Project Scope
- ✅ Target Users
- ✅ Functional Requirements
- ✅ Non-Functional Requirements
- ✅ Use Cases
- ✅ Constraints
- ✅ Assumptions

Today we will learn:

- What is System Design?
- What is System Architecture?
- Components of an AI/ML application
- Frontend and Backend communication
- Database integration
- AI/ML model integration
- API flow
- Architecture diagrams
- Designing the architecture for the final project

📌 Part 2: What is System Design?

Definition

System Design is the process of deciding how different components of a software application will be organized and how they will communicate with each other.

Simple Definition

Before building a house, we create a plan.

Similarly, before developing software, we create a **system design**.

📌 Part 3: What is System Architecture?

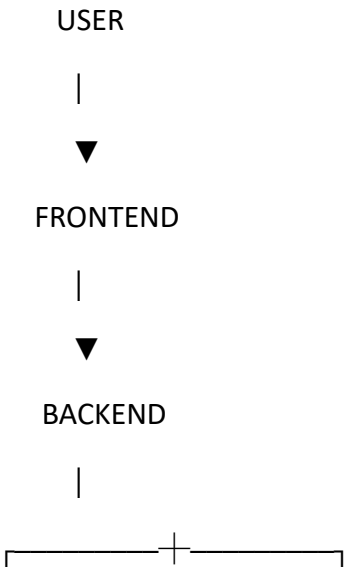
System architecture shows the **overall structure of the application**.

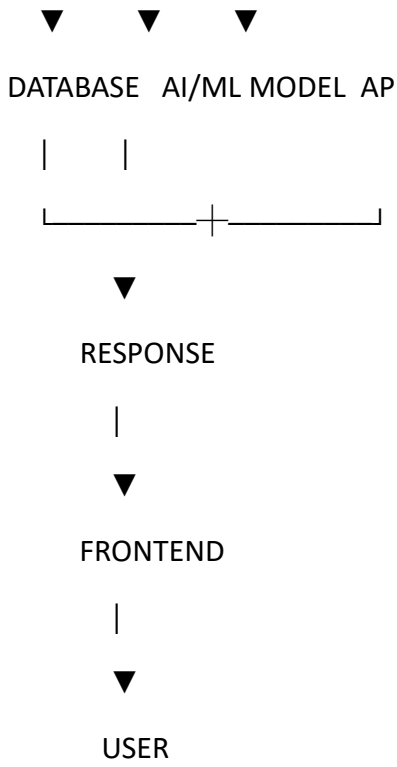
It answers questions such as:

- What components are required?
- Where is the data stored?
- Where does the AI model run?
- How does the frontend communicate with the backend?
- How does the backend communicate with the database?
- How does the backend send data to the AI model?

📌 Part 4: Basic AI/ML Application Architecture

A simple AI application can have:





📌 Part 5: Main Components

1. Frontend

The frontend is the part of the application that users interact with.

Examples:

- HTML
- CSS
- Bootstrap
- JavaScript
- React
- Flutter

Example

For an AI chatbot, the frontend may contain:

- Message box
- Send button
- Chat history

- File upload button
- Response area

📌 Part 6: Backend

The backend handles the application's main logic.

For your projects, you can use:

Python + FastAPI

The backend can:

- Receive user requests
- Validate input
- Communicate with AI models
- Access databases
- Process files
- Return responses

📌 Part 7: Database

A database stores application information.

Examples:

- SQLite
- MySQL
- PostgreSQL

Depending on the project, you may store:

- User accounts
- User information
- Chat history
- Uploaded file information
- Prediction results
- Application settings

📌 Part 8: AI/ML Model

The AI/ML component performs the intelligent part of the application.

Examples:

Computer Vision

Image → YOLO → Object Detection

Machine Learning

Input Data → ML Model → Prediction

NLP

Text → NLP Model → Classification / Extraction

RAG

Question

↓

Retriever

↓

Relevant Documents

↓

LLM

↓

Answer

📌 Part 9: API

What is an API?

API stands for **Application Programming Interface**.

It allows different software components to communicate with each other.

Example

The frontend sends:

"What is Machine Learning?"

to the backend.

The backend processes the request and returns:

"Machine Learning is a branch of Artificial Intelligence..."

📌 Part 10: Frontend → Backend Flow

Suppose we have an AI prediction application.

The user enters:

Age = 25

Experience = 2 years

The frontend sends this information to the backend.

User

↓

Frontend

↓

FastAPI

↓

ML Model

↓

Prediction

↓

FastAPI

↓

Frontend

↓

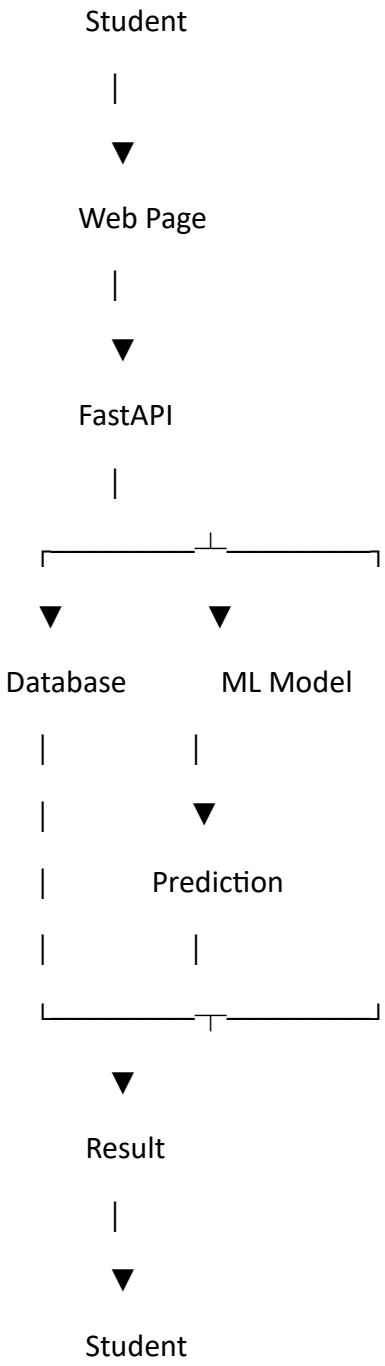
User

📌 Part 11: AI/ML Project Architecture Example

Suppose the project is:

AI Student Performance Predictor

Architecture:



📌 Part 12: RAG Project Architecture

Since interns have already learned RAG, they can use the following architecture for a RAG-based final project:

USER


FRONTEND
FASTAPI
USER QUESTION
EMBEDDING
VECTOR DATABASE
RELEVANT CHUNKS
LLM
GENERATED ANSWER
FRONTEND

📌 Part 13: Document Upload Flow in RAG

When a user uploads a PDF:

PDF Upload

↓

FastAPI

↓

Document Loader

↓

Text Extraction

↓

Text Chunking

↓

Embedding Model

↓

Vector Database

Later, when the user asks a question:

Question

↓

Embedding

↓

Vector Database

↓

Relevant Chunks

↓

LLM

↓

Answer

📌 Part 14: Database Flow

Example:

User

↓

Login

↓

FastAPI

↓

Database

↓

Verify User

↓

Login Success

For a chatbot:

User Question

↓

FastAPI

↓

Chatbot

↓

Response

↓

Database

↓

Chat History

📌 Part 15: Why Architecture is Important

A good architecture helps us:

✅ Understand the complete application.

✅ Divide the project into modules.

✅ Assign tasks to team members.

✅ Identify dependencies.

✅ Make debugging easier.

✅ Make future improvements easier.

✅ Plan development before coding.

📌 Part 16: Modular Architecture

Instead of putting everything into one Python file, divide the project into modules.

Example:

project/

|

├── main.py

├── routes/

├── models/

├── services/

├── database/

├── ai/

├── utils/

├── templates/

├── static/

└── uploads/

Each folder has a specific responsibility.

📌 Part 17: Example Responsibilities

routes/

Handles API endpoints.

models/

Contains database models.

services/

Contains business logic.

ai/

Contains AI/ML-related code.

database/

Handles database connections.

templates/

Contains HTML pages.

static/

Contains CSS, JavaScript, and other static files.

📌 Part 18: System Design vs SRS

SRS	System Design
Describes requirements	Describes implementation structure
What should the system do?	How will the system work?
Focuses on requirements	Focuses on architecture
Prepared before development	Prepared before/during development

Easy way to remember:

SRS → WHAT

System Design → HOW

📌 Part 19: Common Architecture Mistakes

❌ Starting development without understanding requirements.

❌ Putting all code in one file.

❌ Mixing frontend and backend logic unnecessarily.

❌ Hardcoding API keys.

❌ Not planning database structure.

❌ Not defining how components communicate.

📌 Part 20: Theory Questions

1. What is system design?

System design is the process of defining the structure, components, interactions, and communication mechanisms of a software system.

2. What is system architecture?

System architecture represents the overall structure of an application and the relationships between its components.

3. What is the role of a backend?

The backend handles application logic, processes requests, communicates with databases and AI models, and sends responses to the frontend.

4. What is an API?

An API is an interface that allows different software components to communicate with each other.

5. Why is modular architecture useful?

It separates the application into manageable components, making development, testing, debugging, and maintenance easier.

📌 Part 21: Interview Questions

What is the difference between SRS and system design?

SRS describes **what the system should do**, while system design explains **how the system will be implemented**.

Explain a typical AI application architecture.

A typical AI application may contain a frontend, backend, database, AI/ML model, and external APIs. The frontend communicates with the backend, while the backend handles business logic and communicates with the database and AI components.

Why should frontend and backend be separated?

Separation makes the application easier to develop, test, maintain, and scale.

Why is API important in an AI application?

The API acts as a communication layer between the frontend and backend and allows the application to send inputs and receive AI-generated results.

🎯 Tasks for Day 83

🧠 Theory

1. Define system design.
2. Define system architecture.
3. Explain the role of frontend, backend, database, and AI model.
4. What is an API?
5. Differentiate SRS and system design.

💻 Practical Task 1 – Architecture Diagram

Create a complete architecture diagram for your final project.

It should include:

- User
- Frontend
- Backend
- Database
- AI/ML model
- External APIs, if required

💻 Practical Task 2 – Data Flow

Draw the complete flow of your application.

For example:

User

↓

Frontend

↓

FastAPI

↓

AI Model

↓

Database

↓

Response

↓

Frontend

Modify this according to your project.

💻 **Practical Task 3 – Project Structure**

Create the initial folder structure for your final project.

For example:

AI_Project/

|

├── main.py

├── routes/

├── models/

├── services/

├── ai/

├── database/

├── templates/

├── static/

└── uploads/

💻 Practical Task 4 – Module Identification

Identify the major modules in your project.

Example:

AI Resume Screening System

1. User Management
2. Resume Upload
3. Resume Parser
4. Skill Extraction
5. Candidate Scoring
6. Database
7. Dashboard
8. Report Generation

💻 Practical Task 5 – Team Responsibility

If you are working in a team, assign responsibilities.

Team Member	Responsibility
Member 1	Frontend
Member 2	Backend
Member 3	AI/ML
Member 4	Database
Member 5	Testing & Documentation